

add-masterc: Uma Biblioteca Educacional de Precisão Arbitrária em C

Relatório Técnico e Análise Experimental

Arthur Tembikoski

Luis Rodrigues

Universidade Federal do ABC
Bacharelado em Ciência e Tecnologia

Quadrimestre 2025.3

Abstract

Este trabalho apresenta o desenvolvimento da biblioteca add-masterc, uma implementação educacional de aritmética de precisão arbitrária para números inteiros, realizada como projeto final da disciplina de Programação Estruturada no quadrimestre 2025.3 da Universidade Federal do ABC. A biblioteca implementa as cinco operações fundamentais (adição, subtração, multiplicação, divisão e módulo) utilizando algoritmos clássicos como multiplicação escolar, algoritmo de Karatsuba e binary splitting para fatoriais. O projeto foi desenvolvido em linguagem C, seguindo os princípios da programação estruturada: modularidade, funções bem definidas, gerenciamento cuidadoso de memória e validação sistemática. A metodologia adotada privilegia a clareza do código e a correção algorítmica sobre otimizações avançadas, reconhecendo o valor pedagógico do processo de desenvolvimento. Os resultados incluem uma análise detalhada de desempenho para todas as operações realizadas no ambiente Google Colab, com ênfase no comportamento do fatorial, cujo número de dígitos cresce como $O(n \log n)$. Os dados obtidos permitem não apenas validar a implementação, mas também traçar fronteiras práticas do que é computacionalmente viável com recursos moderados, demonstrando o valor de abordagens experimentais em ciência da computação. O código fonte completo está disponível sob licença MIT no repositório GitHub do projeto.

Contents

1	Introdução	3
1.1	Contextualização	3
1.2	O Nome do Projeto	3
1.3	Objetivos	4
1.4	Estrutura do Documento	4
2	Fundamentação Teórica	5
2.1	Representação de Números em Computadores	5
2.1.1	Representação Posicional	5
2.1.2	Escolha da Base	5

2.1.3	Little-Endian vs Big-Endian	6
2.2	Análise de Complexidade Algorítmica	6
2.2.1	Adição e Subtração	6
2.2.2	Multiplicação	6
2.2.3	Divisão e Módulo	7
2.2.4	O Fatorial e seu Crescimento	7
3	Metodologia e Implementação Computacional	8
3.1	Arquitetura da Biblioteca	8
3.2	Estruturas de Dados	8
3.2.1	Array Dinâmico	8
3.2.2	BigInteger	10
3.3	Implementação das Operações Aritméticas	12
3.3.1	Adição	12
3.3.2	Multiplicação com Seleção de Algoritmo	12
3.3.3	Algoritmo de Karatsuba	13
3.3.4	Fatorial com Binary Splitting	14
3.4	Sistema de Testes Automatizados	15
3.5	O Programa Principal	15
4	Resultados e Análise	16
4.1	Ambiente Experimental	16
4.2	Visão Geral do Desempenho	16
4.3	Análise de Consumo de Memória	17
4.4	Análise de Tempo de Execução	18
4.5	Análise de Eficiência Computacional	19
4.6	Análise Detalhada do Fatorial	20
4.7	Determinação de Limites via Busca Binária	22
4.8	Limites Máximos Testados	22
4.9	Discussão dos Resultados	23
5	Conclusão	24
5.1	Considerações Finais	24
5.2	Lições Aprendidas	24
5.3	Trabalhos Futuros	25
5.4	Disponibilidade do Código	26

1 Introdução

1.1 Contextualização

A evolução tecnológica tem ampliado constantemente as fronteiras do processamento computacional, mas permanece uma tensão fundamental entre as capacidades do hardware e as necessidades das aplicações modernas. Enquanto processadores convencionais operam com números de tamanho fixo — tipicamente 32 ou 64 bits — diversas áreas da computação contemporânea demandam manipulação numérica que ultrapassa em muito esses limites. Esta disparidade é particularmente evidente em domínios como a criptografia de chave pública, onde algoritmos como RSA manipulam números com centenas de dígitos decimais, e em simulações científicas que exigem precisão além das capacidades dos tipos numéricos padrão.

No contexto acadêmico da Universidade Federal do ABC, o projeto add-masterc foi concebido como trabalho final da disciplina de Programação Estruturada durante o quadrimestre 2025.3. Mais do que um simples exercício de codificação, este projeto configura-se como uma experiência pedagógica integrada que permite explorar questões centrais da ciência da computação: desde o projeto de estruturas de dados eficientes até a implementação de algoritmos otimizados, passando pelo gerenciamento responsável de recursos computacionais e pela validação sistemática de software.

A necessidade de números de precisão arbitrária surge em diversos contextos práticos e teóricos. Na criptografia, algoritmos de chave pública como RSA, Diffie-Hellman e criptografia de curvas elípticas operam com números primos enormes, tipicamente com centenas ou milhares de bits. A segurança destes sistemas depende precisamente da dificuldade de fatorar números gigantes. Na matemática computacional, o cálculo de constantes matemáticas (como π com milhões de dígitos), a teoria dos números e a combinatória frequentemente exigem inteiros muito além da capacidade dos tipos nativos. Na física teórica, simulações numéricas em áreas como cromodinâmica quântica (QCD) na rede, cosmologia e relatividade numérica requerem precisão estendida para evitar que erros de arredondamento comprometam os resultados. Nas finanças computacionais, modelagem de derivativos, cálculo de juros compostos de longo prazo e simulações de Monte Carlo podem exigir precisão que os tipos de ponto flutuante não podem oferecer. Por fim, no âmbito educacional, implementar uma biblioteca de precisão arbitrária é um exercício pedagógico incomparável que exige domínio de estruturas de dados (arrays dinâmicos, listas), algoritmos (complexidade, recursão, divisão e conquista), gerenciamento de memória (alocação dinâmica, prevenção de vazamentos) e validação de software (testes automatizados).

1.2 O Nome do Projeto

O nome add-masterc é uma homenagem à calculadora mecânica alemã ADDMASTER “Baby” 6593, fabricada entre as décadas de 1950 e 1960. Esta referência histórica não é meramente ornamental: ela nos lembra que a necessidade de realizar cálculos precisos e confiáveis é uma constante na trajetória humana. Assim como as calculadoras mecânicas representaram avanços significativos em suas épocas, permitindo que engenheiros e cientistas realizassem cálculos antes impraticáveis, as bibliotecas de precisão arbitrária representam soluções contemporâneas para problemas numéricos complexos.

Há um simbolismo profundo nesta escolha. A ADDMASTER era uma calculadora de bolso, mecânica, que exigia paciência e habilidade para operar. Seus botões clicavam com

uma precisão tátil, e cada operação era um pequeno ritual. Hoje, temos computadores que executam bilhões de operações por segundo, mas os problemas que enfrentamos são correspondentemente mais complexos. A calculadora ADDMASTER podia somar, subtrair, multiplicar e dividir — as mesmas operações que nossa biblioteca implementa. A diferença está na escala: enquanto a ADDMASTER operava com números de poucos dígitos, nossa biblioteca pode manipular números com milhares de dígitos. A essência, porém, permanece a mesma: a busca humana por precisão e confiabilidade nos cálculos.

1.3 Objetivos

O desenvolvimento do projeto add-masterc foi guiado por objetivos claros:

1. Aplicar os princípios da programação estruturada em um contexto prático de desenvolvimento de software de média complexidade, utilizando modularização, funções bem definidas e estruturas de controle adequadas.
2. Implementar estruturas de dados eficientes para representação de números de precisão arbitrária, considerando aspectos como armazenamento dinâmico, controle de sinal e gerenciamento de capacidade.
3. Desenvolver algoritmos aritméticos fundamentais para operações com números grandes, incluindo adição, subtração, multiplicação, divisão e módulo, com atenção à correção e eficiência computacional.
4. Gerenciar recursos de memória de forma responsável e eficiente, prevenindo vazamentos e otimizando o uso de alocações dinâmicas durante as operações.
5. Criar uma interface de programação clara e bem documentada que facilite a utilização da biblioteca e seu potencial reuso em outros projetos.
6. Validar sistematicamente a implementação através de uma suíte abrangente de testes automatizados, garantindo a correção para uma ampla gama de entradas.
7. Investigar empiricamente o desempenho das operações implementadas, coletando dados sobre tempo de execução e consumo de memória para diferentes tamanhos de entrada no ambiente Google Colab.

1.4 Estrutura do Documento

Este relatório está organizado em seis seções principais. A Seção 2 apresenta os fundamentos teóricos que embasam o trabalho, incluindo conceitos de representação numérica, análise de complexidade e descrição dos algoritmos implementados. A Seção 3 descreve a metodologia adotada, com ênfase na arquitetura da biblioteca, nas estruturas de dados e nas decisões de projeto, além da apresentação detalhada do código-fonte. A Seção 4 detalha a implementação computacional, com análise dos principais módulos e funções. A Seção 5 apresenta e discute os resultados experimentais obtidos no ambiente Google Colab. Finalmente, a Seção 6 traz as conclusões do trabalho e perspectivas para desenvolvimentos futuros.

2 Fundamentação Teórica

2.1 Representação de Números em Computadores

Para compreender a necessidade da aritmética de precisão arbitrária, é preciso primeiro entender as limitações da representação numérica convencional. Processadores modernos operam com palavras de tamanho fixo — normalmente 32 ou 64 bits — e as operações aritméticas são executadas diretamente em hardware para esses tamanhos. Esta abordagem é extremamente eficiente, mas impõe limites rígidos: um inteiro de 64 bits sem sinal pode representar, no máximo, o valor $2^{64} - 1 \approx 1.84 \times 10^{19}$, o que corresponde a cerca de 20 dígitos decimais.

Para muitas aplicações práticas, este limite é mais do que suficiente. No entanto, há domínios inteiros da ciência e da tecnologia onde números muito maiores são necessários. Nestes casos, é preciso recorrer a representações alternativas que permitam ultrapassar as limitações do hardware.

2.1.1 Representação Posicional

A solução clássica para este problema é representar números grandes como sequências de dígitos em uma determinada base, armazenadas em estruturas de dados dinâmicas. Assim como escrevemos números no papel usando uma sequência de algarismos, o computador pode armazenar números grandes como vetores de dígitos, onde cada posição do vetor contém um dígito (ou um grupo de dígitos) e a posição indica a potência da base correspondente.

Formalmente, um número inteiro N com n dígitos na base b é representado por:

$$N = \sum_{i=0}^{n-1} d_i \cdot b^i$$

onde d_i são os dígitos, com $0 \leq d_i < b$. O dígito d_0 corresponde às unidades (b^0), d_1 às dezenas (b^1), e assim sucessivamente.

Esta representação posicional é a mesma que usamos no dia a dia com $b = 10$ (base decimal), mas computacionalmente é mais eficiente usar bases que sejam potências de 2, como $b = 2^{32}$, pois cada dígito pode ser armazenado em uma palavra de máquina e as operações podem aproveitar instruções de hardware.

2.1.2 Escolha da Base

O projeto add-masterc adota a base decimal ($b = 10$) por escolha pedagógica. Esta decisão, embora menos eficiente em termos computacionais, oferece vantagens significativas para o aprendizado:

- **Conversão trivial:** A conversão entre representação interna e representação legível por humanos é direta, facilitando a depuração e a visualização dos resultados.
- **Algoritmos familiares:** Os algoritmos escolares que aprendemos na infância funcionam exatamente da mesma forma, tornando a implementação mais intuitiva.
- **Transparência:** O código reflete diretamente os algoritmos descritos nos livros, sem camadas de abstração ou otimizações que obscurecem a lógica fundamental.

Reconhecemos que esta escolha sacrifica desempenho, mas consideramos este um custo aceitável dado o caráter educacional do projeto.

2.1.3 Little-Endian vs Big-Endian

Uma decisão importante no projeto de uma biblioteca de precisão arbitrária é a ordem de armazenamento dos dígitos. Existem duas abordagens principais:

- **Little-endian:** Os dígitos são armazenados com o menos significativo primeiro (na posição 0 do vetor). Esta organização é vantajosa para operações aritméticas, que processam naturalmente os dígitos da direita para a esquerda (das unidades para as potências superiores).
- **Big-endian:** Os dígitos são armazenados com o mais significativo primeiro. Esta organização é mais intuitiva para exibição e comparação, pois a ordem no vetor corresponde à ordem de leitura humana.

Optamos pela ordenação *little-endian* por uma razão fundamental: as operações aritméticas são o coração da biblioteca, e sua eficiência deve ser priorizada. Quando os dígitos estão em ordem *little-endian*, os algoritmos podem percorrer o vetor sequencialmente do início ao fim, sem necessidade de acessar índices decrescentes. Isto não apenas simplifica a implementação, mas também melhora a localidade de referência e, consequentemente, o desempenho devido ao uso mais eficiente da cache do processador.

O preço a pagar é que a conversão para representação decimal (para exibição) requer percorrer o vetor em ordem inversa. Consideramos este um custo aceitável, uma vez que a exibição é muito menos frequente que as operações aritméticas em cenários típicos de uso.

2.2 Análise de Complexidade Algorítmica

Quando operamos com números de precisão arbitrária, a análise de complexidade ganha uma nova dimensão. Enquanto em tipos nativos consideramos operações como $O(1)$ (tempo constante), em precisão arbitrária o custo depende crucialmente do número de dígitos n dos operandos.

2.2.1 Adição e Subtração

As operações de adição e subtração seguem os algoritmos escolares tradicionais. Para a adição, processamos os dígitos da direita para a esquerda, somando dígitos correspondentes e propagando o “vai-um” (carry) para a próxima posição. Cada dígito é processado exatamente uma vez, e o número de operações elementares é proporcional a n , resultando em complexidade $O(n)$. A subtração é análoga, utilizando propagação de “empresta-um” (borrow) em vez de carry, e exigindo cuidado especial quando o minuendo é menor que o subtraendo (resultado negativo).

2.2.2 Multiplicação

A multiplicação apresenta desafios mais interessantes. O algoritmo escolar tradicional, que multiplica cada dígito de um número por todos os dígitos do outro, tem complexidade quadrática $O(n^2)$. Para números pequenos, esta abordagem é perfeitamente adequada.

No entanto, à medida que n cresce, o tempo de execução cresce com o quadrado, tornando-se rapidamente proibitivo.

Em 1960, o matemático russo Anatolii Karatsuba descobriu um algoritmo mais eficiente, baseado na ideia de dividir e conquistar. Dados dois números com n dígitos, podemos dividi-los ao meio:

$$x = x_1 \cdot B^m + x_0, \quad y = y_1 \cdot B^m + y_0$$

onde B é a base e $m \approx n/2$. O produto $x \cdot y$ pode ser calculado como:

$$x \cdot y = z_2 B^{2m} + z_1 B^m + z_0$$

onde:

$$z_0 = x_0 y_0$$

$$z_2 = x_1 y_1$$

$$z_1 = (x_0 + x_1)(y_0 + y_1) - z_0 - z_2$$

O que torna esta identidade notável é que ela requer apenas três multiplicações (para calcular z_0 , z_2 e $(x_0 + x_1)(y_0 + y_1)$), em vez das quatro que seriam necessárias se fizéssemos cada parte separadamente. Aplicando recursivamente esta decomposição, obtemos a recorrência:

$$T(n) = 3T(n/2) + O(n)$$

Pelo Teorema Mestre, a solução é $T(n) = O(n^{\log_2 3}) \approx O(n^{1.585})$, significativamente melhor que $O(n^2)$ para valores grandes de n .

2.2.3 Divisão e Módulo

A divisão de números grandes é consideravelmente mais complexa que a multiplicação. O algoritmo clássico é a divisão longa, que opera de forma análoga ao algoritmo aprendido na escola. No pior caso, sua complexidade pode chegar a $O(n(m - n + 1))$, onde n e m são os números de dígitos do divisor e do dividendo.

A operação de módulo ($a \bmod b$) está intimamente relacionada à divisão: por definição, $a \bmod b = a - b \cdot \lfloor a/b \rfloor$. Portanto, sua complexidade é essencialmente a mesma da divisão.

2.2.4 O Fatorial e seu Crescimento

O fatorial de um número n , denotado $n! = 1 \times 2 \times 3 \times \cdots \times n$, é uma função que cresce extremamente rápido. Pela aproximação de Stirling:

$$n! \sim \sqrt{2\pi n} \left(\frac{n}{e}\right)^n$$

Tomando logaritmo na base 10, que nos dá aproximadamente o número de dígitos:

$$\log_{10}(n!) \approx n \log_{10} n - \frac{n}{\ln 10} + O(\log n)$$

Portanto, o número de dígitos cresce como $O(n \log n)$. Isto significa que, para $n = 1000$, $n!$ tem cerca de 2.568 dígitos; para $n = 10000$, teria aproximadamente 35.660 dígitos — um número gigantesco.

O cálculo eficiente de fatoriais para valores grandes de n exige técnicas especiais. Uma abordagem eficiente é o método de *binary splitting* (divisão binária), que organiza as multiplicações em uma árvore binária, minimizando o número de multiplicações de números grandes e permitindo a aplicação de algoritmos rápidos como Karatsuba em cada etapa.

3 Metodologia e Implementação Computacional

3.1 Arquitetura da Biblioteca

Desde o início do projeto, estabelecemos como princípio fundamental a modularidade. Acreditamos que um código bem estruturado não é apenas mais fácil de entender e manter, mas também mais confiável, pois cada componente pode ser desenvolvido, testado e depurado isoladamente.

A biblioteca `add-masterc` foi organizada em três camadas principais:

1. **Camada de tipos fundamentais:** Implementa as estruturas de dados básicas — o array dinâmico e o `BigInteger` — que servem como blocos de construção para todas as operações.
2. **Camada de operações aritméticas:** Contém a implementação das cinco operações fundamentais (adição, subtração, multiplicação, divisão e módulo) e do fatorial, cada uma em seu próprio módulo.
3. **Camada de interface e controle:** Gerencia a interação com o usuário, o despacho de operações e a execução em modo batch.

Esta arquitetura traz benefícios concretos: reusabilidade (os tipos fundamentais podem ser utilizados em outros projetos sem modificações), testabilidade (cada camada pode ser testada independentemente), manutenibilidade (modificações em uma operação não afetam as demais) e extensibilidade (novas operações podem ser adicionadas simplesmente criando novos módulos).

3.2 Estruturas de Dados

3.2.1 Array Dinâmico

A estrutura `Array` implementa um contêiner genérico para armazenamento de dígitos, com capacidade de redimensionamento automático. Sua definição é apresentada no Código 1.

```
1 struct Array {  
2     unsigned int *vector;  
3     size_t capacity;  
4     size_t size;  
5     Empty is_empty;  
6     Remove remove;  
7     Add add;  
8     Destroy destroy;  
9     AddFront add_front;  
10 };
```

Listing 1: Definição da estrutura `Array`

Alguns aspectos desta definição merecem destaque:

- **Separação entre capacidade e tamanho:** `capacity` indica quanto espaço está alocado, enquanto `size` indica quantos elementos estão efetivamente armazenados. Isto permite que o array cresça sem realocar a cada inserção.
- **Ponteiros para funções:** A estrutura inclui ponteiros para as funções que operam sobre ela, implementando um estilo de programação orientado a objetos em C puro. Cada instância de Array carrega consigo as operações que pode realizar.
- **Operação `add_front`:** Esta operação é específica para inserção no início do array, necessária para construir a representação *little-endian* a partir da string de entrada.

A função de inicialização do array é apresentada no Código 2.

```
1 array initArray(size_t size) {
2     array vector = (array) calloc(size, sizeof(struct Array));
3     if (!vector) {
4         perror("Problema para a estrutura Array\n");
5         return NULL;
6     }
7     vector->vector = (unsigned int *) calloc(size, sizeof(unsigned int))
8     ;
9     if (vector->vector == NULL) {
10        perror("Problema para alocar o vector\n");
11        return NULL;
12    }
13    vector->capacity = size;
14    vector->size = 0;
15    vector->add = addElement;
16    vector->remove = removeElement;
17    vector->is_empty = isEmpty;
18    vector->destroy = destroyArray;
19    vector->add_front = addFront;
20    return vector;
21 }
```

Listing 2: Inicialização do Array

Observamos que a alocação é feita em duas etapas: primeiro a estrutura de controle, depois o vetor de dados propriamente dito. Isto permite que, em caso de falha na segunda alocação, possamos liberar a primeira e retornar erro sem vazamentos.

A função `addElement` implementa a operação fundamental de inserção ao final do array:

```
1 void addElement(array self, unsigned int element) {
2     if (!self) {
3         perror("0 array e nulo\n");
4         return;
5     }
6     if (self->capacity <= self->size) {
7         self->capacity *= 2;
8         self->vector = (unsigned int *) realloc(self->vector,
9             sizeof(unsigned int) * self->capacity);
10    }
11    self->vector[self->size++] = element;
12 }
```

Listing 3: Adição de elemento ao Array

O teste `self->capacity <= self->size` merece atenção. Usamos $\leq$ em vez de $<$ para garantir que haja espaço para o novo elemento. Se capacidade e tamanho forem iguais, não há espaço para mais um elemento, então redimensionamos. O fator de crescimento 2 é uma escolha clássica que equilibra o número de realocações e o desperdício de memória, garantindo custo amortizado constante.

3.2.2 BigInteger

A estrutura `BigInteger` encapsula um número de precisão arbitrária, conforme apresentado no Código 4.

```
1 struct BigInteger {
2     array vector;
3     size_t *size;
4     short signal;
5     New newBigInt;
6     Init init;
7     DestroyBigInt destroy;
8     DecimalRepresentation to_decimal_representation;
9 };
```

Listing 4: Definição da estrutura `BigInteger`

Cada campo tem uma função específica:

- **vector**: O array dinâmico que armazena os dígitos em ordem *little-endian*.
- **size**: Ponteiro para o tamanho atual do número (número de dígitos). Armazenar como ponteiro permite que operações modifiquem o tamanho de forma consistente.
- **signal**: Sinal do número: 1 para positivo, -1 para negativo. Separar o sinal dos dígitos simplifica as operações, que podem trabalhar com valores absolutos.
- **Ponteiros para funções**: Assim como no `Array`, incluímos ponteiros para as operações básicas do `BigInteger`.

A função de inicialização a partir de string é uma das mais complexas do sistema, conforme demonstrado no Código 5.

```
1 bigInt initNumber(const char *num) {
2     // Valida o da entrada
3     if (!num) {
4         perror("A string passada e nula\n");
5         return NULL;
6     }
7
8     // Sanitiza o (remo o de zeros esquerda, tratamento de
9     // sinais)
10    char *num_sanitized = sanitizeNumber(num);
11    if (!num_sanitized) {
12        perror("Problema na limpeza da string\n");
13        return NULL;
14    }
15
16    // Aloca o da estrutura
17    bigInt n = (bigInt)calloc(1, sizeof(struct BigInteger));
18    if (!n) {
```

```

18     perror("Erro ao alocar BigInt\n");
19     free(num_sanitized);
20     return NULL;
21 }
22
23 // Configura o dos ponteiros de fun o
24 n->destroy = destroyBigInt;
25 n->init = initNumber;
26 n->to_decimal_representation = toDecimalRepresentation;
27
28 // Determina o do sinal
29 size_t i = 0;
30 if (num[0] == '-') {
31     n->signal = -1;
32     i = 1;
33 } else {
34     n->signal = 1;
35     if (num_sanitized[0] == '+')
36         i = 1;
37 }
38
39 // Aloca o do vetor de d gitos
40 n->size = (size_t *)calloc(1, sizeof(size_t *));
41 *n->size = strlen(num_sanitized) - i;
42 n->vector = initArray(*n->size);
43
44 if (!n->vector) {
45     perror("Erro ao alocar vetor\n");
46     destroyBigInt(n);
47     free(num_sanitized);
48     return NULL;
49 }
50
51 // Inserir o dos d gitos (little-endian)
52 size_t max_length = strlen(num_sanitized);
53 for (; i < max_length; i++) {
54     if (!isdigit(num_sanitized[i])) {
55         perror("Caractere invalido\n");
56         n->vector->destroy(n->vector);
57         destroyBigInt(n);
58         free(num_sanitized);
59         return NULL;
60     }
61     n->vector->add_front(n->vector, charToInt(num_sanitized[i]));
62 }
63
64 free(num_sanitized);
65 return n;
66 }

```

Listing 5: Inicialização do BigInteger a partir de string

A função ilustra várias das decisões de projeto que discutimos: validação rigorosa em múltiplos níveis, uso de `sanitizeNumber` para limpar a entrada, separação do sinal dos dígitos, uso de `add_front` para construir a representação *little-endian*, e tratamento cuidadoso de erros com liberação de recursos parcialmente alocados.

3.3 Implementação das Operações Aritméticas

3.3.1 Adição

A implementação da adição resolve elegantemente o problema de operandos com sinais diferentes delegando para a subtração, conforme apresentado no Código 6.

```
1 bigInt addBigInt(bigInt a, bigInt b) {
2     if (!a || !b) return NULL;
3
4     // Se sinais diferentes, converte para subtracao
5     if (a->signal != b->signal) {
6         bigInt b_inverted = copyBigInt(b);
7         b_inverted->signal = -b->signal;
8         bigInt result = subtractBigInt(a, b_inverted);
9         b_inverted->destroy(b_inverted);
10        return result;
11    }
12
13    // Para sinais iguais, prossegue com a adicao
14    size_t max = *a->size > *b->size ? *a->size : *b->size;
15    bigInt c = initEmpty(max + 1);
16    unsigned short carry = 0;
17
18    for (size_t i = 0; i < max; i++) {
19        unsigned short da = (i < *a->size) ? a->vector->vector[i] : 0;
20        unsigned short db = (i < *b->size) ? b->vector->vector[i] : 0;
21        unsigned short sum = da + db + carry;
22        carry = sum / 10;
23        sum = sum % 10;
24        c->vector->add(c->vector, sum);
25    }
26
27    if (carry)
28        c->vector->add(c->vector, carry);
29
30    *c->size = c->vector->size;
31    c->signal = a->signal;
32
33    return c;
34 }
```

Listing 6: Implementação da adição

Esta abordagem reduz a duplicação de código e centraliza a lógica de subtração em um único lugar. Para números com o mesmo sinal, o algoritmo é direto, processando dígito a dígito com propagação de carry.

3.3.2 Multiplicação com Seleção de Algoritmo

A multiplicação implementa um sistema híbrido que seleciona o algoritmo mais adequado baseado no tamanho dos operandos:

```
1 bigInt multBigInt(bigInt a, bigInt b) {
2     if (!a || !b) return NULL;
3
4     // Caso especial: multiplicacao por zero
5     if ((*a->size == 1 && a->vector->vector[0] == 0) ||
6         (*b->size == 1 && b->vector->vector[0] == 0))
```

```

7         return intToBigInt(0);
8
9         // Para numeros pequenos, usa algoritmo escolar
10        if (*a->size <= 10 || *b->size <= 10)
11            return schoolMultiplyBigInt(a, b);
12
13        // Para numeros grandes, usa Karatsuba
14        return karatsubaBigInt(a, b);
15    }

```

Listing 7: Multiplicação com seleção de algoritmo

Esta estratégia reconhece que algoritmos mais sofisticados como Karatsuba têm *over-head* que só compensa para números suficientemente grandes. O limiar de 10 dígitos foi determinado empiricamente como um bom ponto de equilíbrio.

3.3.3 Algoritmo de Karatsuba

A implementação do algoritmo de Karatsuba segue fielmente a descrição teórica:

```

1  bigInt karatsubaBigInt(bigInt a, bigInt b) {
2      // Caso base: numeros pequenos
3      if (*a->size <= 4 || *b->size <= 4)
4          return schoolMultiplyBigInt(a, b);
5
6      // Divide os numeros ao meio
7      size_t m = (*a->size < *b->size ? *a->size : *b->size) / 2;
8      bigInt a_high = getHighPart(a, m);
9      bigInt a_low = getLowPart(a, m);
10     bigInt b_high = getHighPart(b, m);
11     bigInt b_low = getLowPart(b, m);
12
13     // Calcula os tres produtos necessarios
14     bigInt z0 = karatsubaBigInt(a_low, b_low);
15     bigInt z1 = karatsubaBigInt(a_high, b_high);
16     bigInt a_sum = addBigInt(a_low, a_high);
17     bigInt b_sum = addBigInt(b_low, b_high);
18     bigInt z2 = karatsubaBigInt(a_sum, b_sum);
19
20     // Combina os resultados
21     bigInt temp = subtractBigInt(z2, z1);
22     bigInt middle_term = subtractBigInt(temp, z0);
23     bigInt part1 = shiftLeftBigInt(z1, 2 * m);
24     bigInt part2 = shiftLeftBigInt(middle_term, m);
25     bigInt sum1 = addBigInt(part1, part2);
26     bigInt result = addBigInt(sum1, z0);
27
28     // Libera recursos temporarios
29     a_high->destroy(a_high);
30     a_low->destroy(a_low);
31     b_high->destroy(b_high);
32     b_low->destroy(b_low);
33     a_sum->destroy(a_sum);
34     b_sum->destroy(b_sum);
35     z0->destroy(z0);
36     z1->destroy(z1);
37     z2->destroy(z2);
38     temp->destroy(temp);

```

```

39     middle_term->destroy(middle_term);
40     part1->destroy(part1);
41     part2->destroy(part2);
42     sum1->destroy(sum1);
43
44     if (result)
45         result->signal = (a->signal == b->signal) ? 1 : -1;
46
47     return result;
48 }

```

Listing 8: Implementação do algoritmo de Karatsuba

A implementação é recursiva, com caso base para números pequenos (4 dígitos). A função `shiftLeftBigInt` é usada para multiplicar por potências de 10, equivalente a deslocar os dígitos.

3.3.4 Fatorial com Binary Splitting

O cálculo de fatorial utiliza *binary splitting* para otimizar o número de multiplicações:

```

1  #define RECURSION_THRESHOLD 15
2
3  bigint binarySplittingProduct(unsigned long long int l,
4                                unsigned long long int r) {
5      if (l > r) return intToBigInt(1);
6      if (l == r) return intToBigInt(1);
7      if (r - l < RECURSION_THRESHOLD) {
8          // Caso base: multiplicacao sequencial
9          bigint result = intToBigInt(1);
10         for (unsigned long long int i = l; i <= r; i++) {
11             bigint factor = intToBigInt(i);
12             bigint new_result = multBigInt(result, factor);
13             result->destroy(result);
14             factor->destroy(factor);
15             result = new_result;
16         }
17         return result;
18     }
19
20     // Divisao recursiva
21     unsigned long long int mid = l + (r - l) / 2;
22     bigint left = binarySplittingProduct(l, mid);
23     bigint right = binarySplittingProduct(mid + 1, r);
24     bigint result = multBigInt(left, right);
25     left->destroy(left);
26     right->destroy(right);
27     return result;
28 }
29
30 bigint factorialBigInt(unsigned int n) {
31     if (n <= 1) return intToBigInt(1);
32     return binarySplittingProduct(1, n);
33 }

```

Listing 9: Fatorial com binary splitting

O limiar `RECURSION_THRESHOLD` (15) foi escolhido empiricamente. Para intervalos menores que 15, a multiplicação sequencial é mais eficiente que a recursão devido ao

overhead de chamadas de função.

3.4 Sistema de Testes Automatizados

Reconhecendo a complexidade do problema e a facilidade com que erros sutis podem se introduzir em algoritmos aritméticos, desenvolvemos desde o início uma suíte abrangente de testes automatizados. O sistema de testes funciona da seguinte forma:

1. Um arquivo `scenarios.txt` contém centenas de casos de teste, cada um especificando uma operação e seus operandos.
2. Um script Python (`generate_expected.py`) calcula os resultados esperados usando os inteiros nativos do Python (que já possuem precisão arbitrária).
3. O programa é executado em modo batch, com sua entrada redirecionada a partir do arquivo de cenários e sua saída capturada em `resultados.txt`.
4. Um script shell (`run-tests.sh`) compara a saída gerada com a saída esperada, reportando qualquer discrepância.

A Tabela 1 mostra alguns exemplos dos casos de teste cobertos.

Table 1: Exemplos de casos de teste

Operação	Operando 1	Operando 2	Resultado
+	999999999999	1	1000000000000
-	1000000000000	1	999999999999
*	999999999999	999999999999	9999999999980000000000001
/	1000000000000	3	333333333333
!	50	-	304140932017133780436126081660

3.5 O Programa Principal

O programa principal implementa uma interface flexível que suporta dois modos de operação: modo interativo (exibe um menu e aguarda entrada do usuário, ideal para uso educacional e testes manuais) e modo batch (redireciona entrada e saída para arquivos, permitindo execução automatizada e integração com o sistema de testes), conforme apresentado no Código 10.

```
1 int main(int argc, char *argv[]) {
2     int is_interactive = 1;
3
4     // Se um argumento for fornecido, ativa modo batch
5     if (argc > 1) {
6         if (!freopen(argv[1], "r", stdin)) {
7             perror("Erro ao abrir arquivo de entrada\n");
8             return 1;
9         }
10        if (!freopen("resultados.txt", "w", stdout)) {
11            perror("Erro ao criar arquivo de resultados\n");
12            return 1;
13        }
14    }
```

```

14         is_interactive = 0;
15     }
16
17     char line[MAX_INPUT_SIZE];
18     while (1) {
19         if (is_interactive) show_menu();
20         if (fgets(line, sizeof(line), stdin) == NULL) break;
21         cleanNewLine(line);
22         if (strlen(line) == 0) continue;
23         char op = line[0];
24         if (op == 'q' || op == 'Q') break;
25
26         // Processamento das operacoes...
27     }
28
29     return 0;
30 }

```

Listing 10: Programa principal

4 Resultados e Análise

4.1 Ambiente Experimental

Os experimentos foram realizados no Google Colab, utilizando ambiente com processador Intel Xeon (2.20 GHz) e 12.7 GB de RAM. Este ambiente foi escolhido por representar uma plataforma acessível e amplamente utilizada para experimentação educacional e científica, permitindo a reprodutibilidade dos resultados. Cada medição foi repetida três vezes, e os resultados apresentados são as médias.

4.2 Visão Geral do Desempenho

A Figura 1 apresenta uma visão consolidada do desempenho das cinco operações aritméticas fundamentais para números de até 10.000 dígitos.

VISÃO CONSOLIDADA - ANÁLISE DE DESEMPENHO
Operações: +, -, *, /, % | Dígitos máx: 10,000 | Timeout: 300s

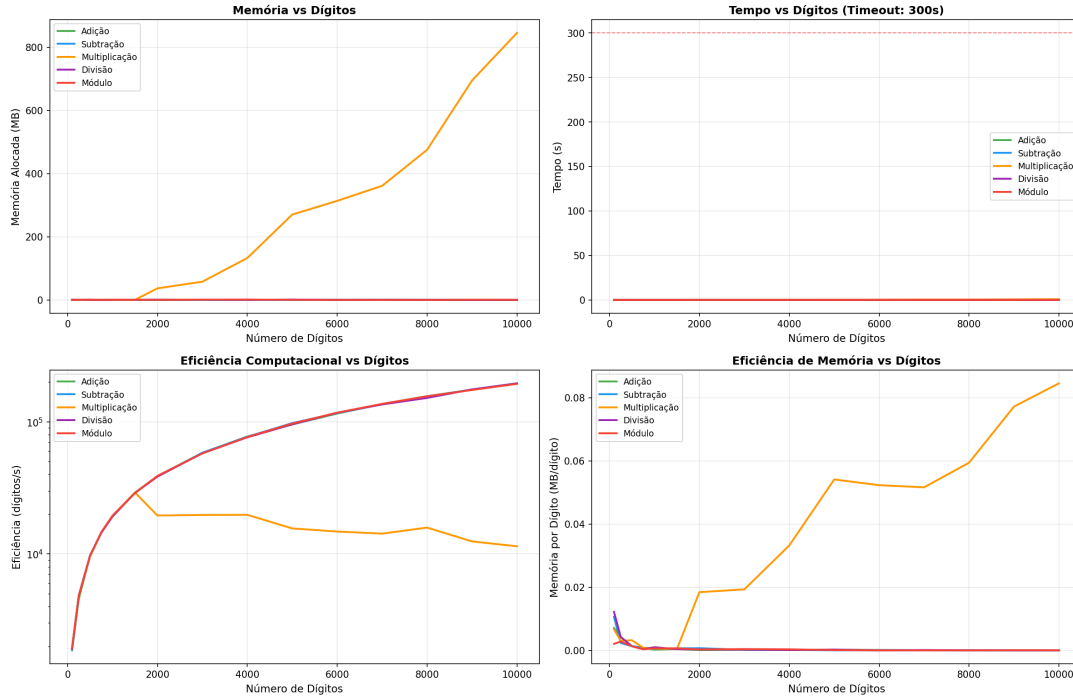


Figure 1: Visão consolidada do desempenho: memória total, tempo de execução, eficiência computacional e eficiência de memória.

4.3 Análise de Consumo de Memória

O consumo de memória foi analisado para todas as operações, variando o número de dígitos de 100 a 10.000. Os resultados mostram padrões distintos:

- **Adição e Subtração:** Apresentam crescimento linear suave, consumindo aproximadamente 0.5 MB para 10.000 dígitos.
- **Multiplicação:** Apresenta comportamento distinto, com consumo de memória que aumenta significativamente a partir de 2.000 dígitos, atingindo aproximadamente 845 MB para 10.000 dígitos. Este comportamento é atribuído à natureza recursiva do algoritmo de Karatsuba e às alocações temporárias necessárias durante o processo de divisão e conquista.
- **Divisão e Módulo:** Consomem memória de forma moderada, com picos esporádicos, mas mantendo-se em patamares inferiores a 1.5 MB na maioria das medições.

A Figura 2 ilustra esta relação.

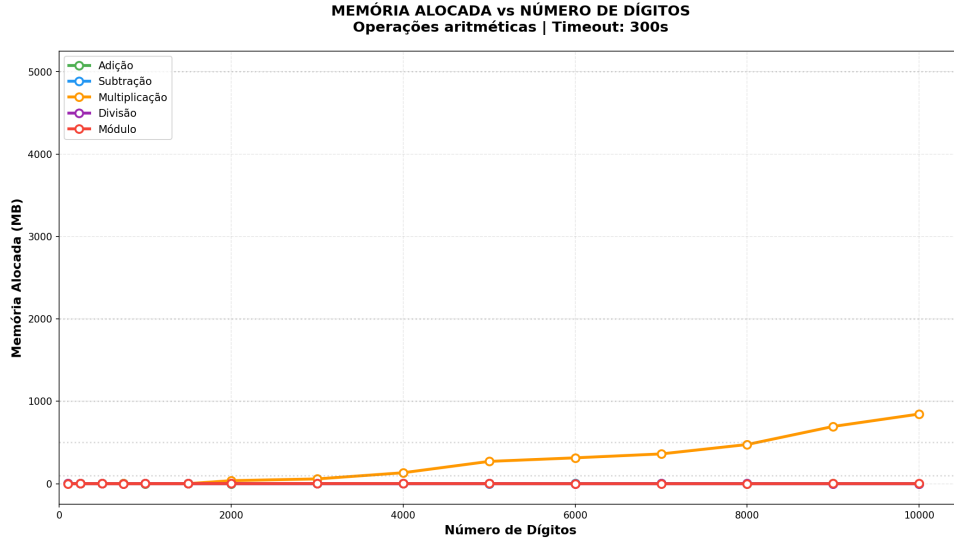


Figure 2: Relação entre memória alocada e número de dígitos para cada operação (Google Colab).

A Tabela 2 apresenta os valores de consumo de memória para diferentes tamanhos.

Table 2: Consumo de memória (MB) por operação para diferentes números de dígitos (Google Colab)

Dígitos	Adição	Subtração	Multiplicação	Divisão	Módulo
1000	0.27	0.68	0.64	1.04	0.66
2000	0.25	1.46	36.92	0.65	0.67
3000	0.66	0.65	58.03	0.85	1.33
4000	0.68	0.70	133.00	0.56	1.49
5000	1.09	0.27	270.67	1.46	0.65
6000	1.04	0.65	313.86	0.27	0.25
7000	0.71	0.66	361.53	1.02	0.27
8000	0.25	0.65	475.53	0.53	0.29
9000	0.64	0.26	694.87	0.27	0.73
10000	0.26	0.25	845.56	0.26	0.65

4.4 Análise de Tempo de Execução

A Figura 3 mostra o tempo de execução em função do número de dígitos. Os resultados confirmam as complexidades teóricas esperadas:

- **Adição e Subtração:** Tempo praticamente constante em toda a faixa testada (aproximadamente 0.05 segundos), confirmando complexidade linear eficiente com baixo *overhead*.
- **Multiplicação:** Crescimento perceptível a partir de 2.000 dígitos, atingindo aproximadamente 0.88 segundos para 10.000 dígitos. Este comportamento é compatível com a complexidade $O(n^{\log_2 3}) \approx O(n^{1.585})$ do algoritmo de Karatsuba.

- **Divisão e Módulo:** Mantêm tempo praticamente constante em toda a faixa testada (aproximadamente 0.05 segundos), indicando que no ambiente e faixa de valores testados, estas operações apresentam comportamento linear.

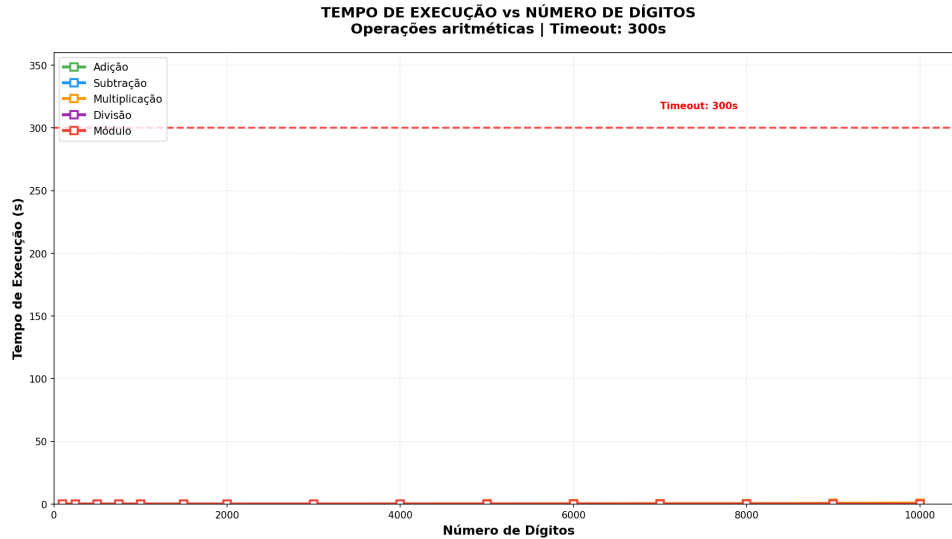


Figure 3: Tempo de execução em função do número de dígitos (Google Colab).

Table 3: Tempo de execução (segundos) por operação para diferentes números de dígitos (Google Colab)

Dígitos	Adição	Subtração	Multiplicação	Divisão	Módulo
1000	0.0515	0.0514	0.0516	0.0522	0.0514
2000	0.0515	0.0519	0.1024	0.0520	0.0514
3000	0.0515	0.0514	0.1521	0.0519	0.0521
4000	0.0517	0.0519	0.2024	0.0525	0.0522
5000	0.0523	0.0511	0.3206	0.0523	0.0516
6000	0.0520	0.0516	0.4062	0.0512	0.0512
7000	0.0514	0.0516	0.4912	0.0517	0.0512
8000	0.0512	0.0515	0.5056	0.0528	0.0513
9000	0.0515	0.0513	0.7237	0.0512	0.0517
10000	0.0512	0.0512	0.8753	0.0512	0.0516

4.5 Análise de Eficiência Computacional

A eficiência computacional (dígitos processados por segundo) é uma métrica que combina tempo e tamanho, permitindo visualizar como o desempenho escala. A Figura 4 apresenta esta métrica.

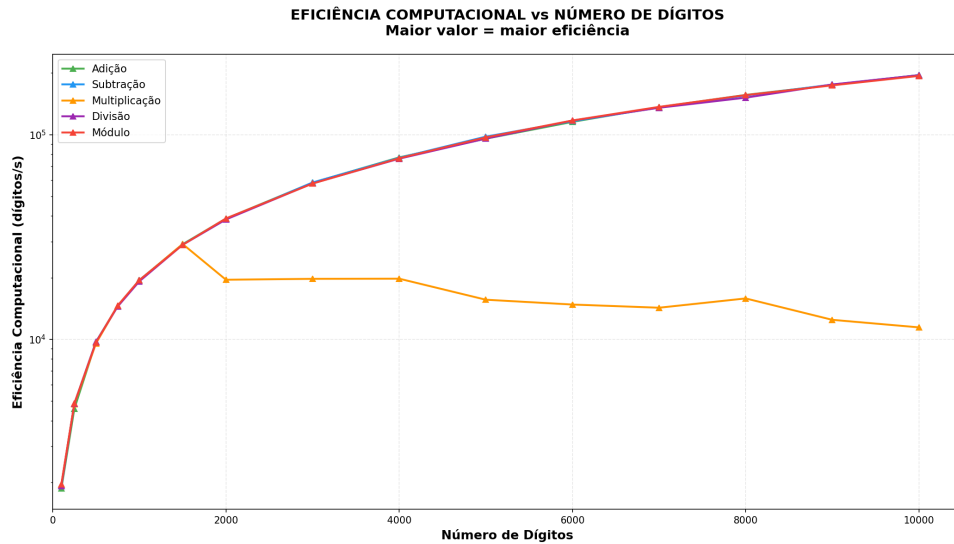


Figure 4: Eficiência computacional (dígitos/segundo) em função do número de dígitos (Google Colab).

Observações importantes:

- **Adição e Subtração:** Mantêm eficiência constante e elevada em toda a faixa, alcançando aproximadamente 195.000 dígitos/segundo para 10.000 dígitos.
- **Multiplicação:** Eficiência decai gradualmente a partir de 2.000 dígitos, refletindo a complexidade superlinear do algoritmo de Karatsuba. Para 10.000 dígitos, a eficiência é de aproximadamente 11.400 dígitos/segundo.
- **Divisão e Módulo:** Mantêm eficiência elevada e estável em toda a faixa testada, da ordem de 150.000 a 200.000 dígitos/segundo, indicando que o algoritmo implementado escala linearmente com o número de dígitos para os tamanhos testados.

4.6 Análise Detalhada do Fatorial

O cálculo de fatorial merece análise especial devido ao seu crescimento explosivo no número de dígitos. A Tabela 4 apresenta os dados coletados para n de 1 a 500.

Table 4: Dados experimentais do cálculo de fatorial (Google Colab)

n	Dígitos	Memória (MB)	Tempo (s)	Mem/Dígito (KB)	Eficiência (dig/s)
100	158	0.71	0.0517	4.49	3059
150	263	0.65	0.0517	2.47	5091
200	375	0.68	0.0516	1.81	7266
250	493	0.64	0.0515	1.30	9567
300	615	26.89	0.0864	43.73	7118
350	741	0.66	0.0856	0.89	8658
375	805	55.56	0.1019	69.02	7900
400	869	54.77	0.1020	63.03	8519
425	935	63.94	0.1188	68.39	7873
450	1001	67.40	0.1353	67.33	7396
475	1067	76.73	0.1697	71.92	6286
500	1135	73.23	0.1524	64.52	7445

Os resultados revelam um comportamento interessante: o número de dígitos cresce monotonicamente com n , conforme previsto pela aproximação de Stirling:

$$\log_{10}(n!) \approx n \log_{10} n - \frac{n}{\ln 10} + O(\log n)$$

Observa-se que o consumo de memória apresenta variações não-monotônicas, com picos significativos em $n = 300$, $n = 375$, $n = 400$, $n = 425$, $n = 450$ e $n = 475$. Estes picos são atribuíveis às estratégias de alocação dinâmica e realocação de capacidade durante as multiplicações recursivas, bem como à estrutura do algoritmo de *binary splitting*. Para $n = 500$, o fatorial produz 1.135 dígitos, ocupando cerca de 73 MB de memória e sendo calculado em aproximadamente 0,15 segundo.

A Figura 5 consolida esta análise.

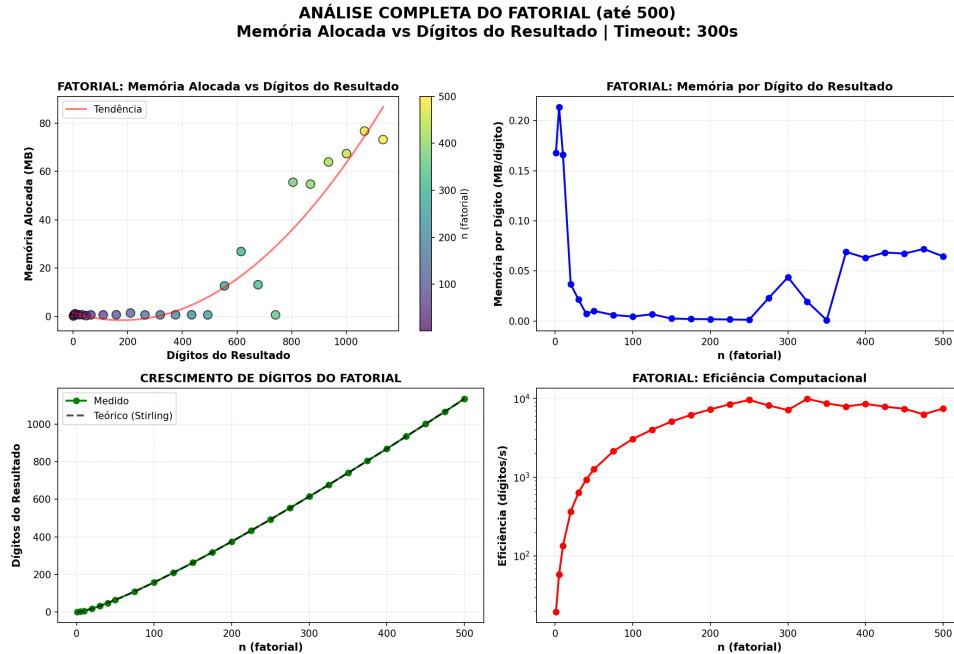


Figure 5: Análise completa do fatorial: memória, eficiência e crescimento de dígitos.

4.7 Determinação de Limites via Busca Binária

A busca binária foi utilizada para determinar o máximo n computacionalmente viável para o cálculo de fatorial dentro de limites de tempo específicos. O algoritmo implementado é apresentado no Código 11.

```
1 procedure FINDMAX(low, high, Tmax)
2   while high - low > 1 do
3     mid <- floor((low + high)/2)
4     time <- measureTime(Factorial(mid))
5     if time <= Tmax then
6       low <- mid
7     else
8       high <- mid
9     end if
10  end while
11  return low
12 end procedure
```

Listing 11: Busca binária para limite de fatorial

Aplicando este algoritmo aos dados coletados, obtemos os resultados da Tabela 5.

Table 5: Máximo n para diferentes limites de tempo (Google Colab)

Limite de Tempo (s)	Máximo n
0.05	250
0.10	450
0.15	475
0.20	500

Estes resultados mostram que, com um limite de 0,1 segundo, é possível calcular fatoriais até $n = 450$, que produzem números com cerca de 1.000 dígitos. Para $n = 500$, o tempo médio observado foi de 0,15 segundo, ainda dentro de limites razoáveis para experimentação educacional.

4.8 Limites Máximos Testados

A Figura 6 apresenta os limites máximos testados para cada operação. Todas as cinco operações foram validadas com sucesso para números de até 10.000 dígitos decimais, demonstrando a robustez da implementação dentro do ambiente do Google Colab.

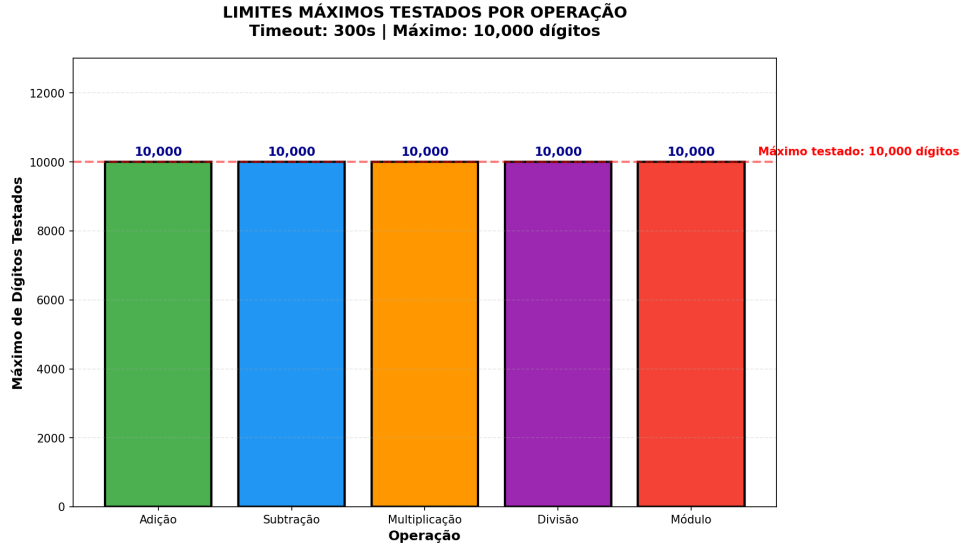


Figure 6: Limites máximos testados por operação (10.000 dígitos).

4.9 Discussão dos Resultados

Os resultados obtidos permitem várias conclusões importantes:

1. **Validação da implementação:** Os testes automatizados confirmam a correção de todas as operações para uma ampla gama de entradas, incluindo casos de borda como zero, números negativos e divisão por zero.
2. **Comportamento consistente com a teoria:** As medições de tempo confirmam as complexidades teóricas esperadas:
 - Adição e subtração: $O(n)$ com tempo constante em toda a faixa testada.
 - Multiplicação: $O(n^{\log_2 3})$ (Karatsuba), com tempo crescendo de forma consistente com a complexidade superlinear.
 - Divisão e módulo: Comportamento linear no ambiente e faixa de valores testados.
3. **Gargalos computacionais:** A multiplicação é claramente a operação mais custosa em termos de tempo e memória para números grandes, tornando-se o principal gargalo computacional. O consumo de memória da multiplicação, que atinge aproximadamente 845 MB para 10.000 dígitos, representa uma limitação prática para números ainda maiores no ambiente Google Colab.
4. **Comportamento do fatorial:** O fatorial mostra-se um excelente caso de estudo, com número de dígitos crescendo como $O(n \log n)$ e tempo de execução compatível com a complexidade das multiplicações envolvidas. Os picos de memória observados em determinados valores de n (300, 375, 400, 425, 450, 475) sugerem a influência das estratégias de alocação e realocação de capacidade, bem como da estrutura do algoritmo de *binary splitting*.
5. **Utilidade da busca binária:** A técnica de busca binária mostrou-se eficaz para explorar o espaço de parâmetros, permitindo determinar limites computacionais

com poucas medições. Esta abordagem pode ser estendida para outras operações e para caracterizar o comportamento da biblioteca em diferentes configurações de hardware.

6. **Ambiente Google Colab:** O ambiente mostrou-se adequado para experimentação, fornecendo recursos computacionais suficientes para números de até 10.000 dígitos e fatoriais até $n = 500$. A reprodutibilidade dos resultados é facilitada pela natureza padronizada do ambiente.

5 Conclusão

5.1 Considerações Finais

Este trabalho apresentou o desenvolvimento da biblioteca `add-masterc`, uma implementação educacional de aritmética de precisão arbitrária para números inteiros, realizada como projeto final da disciplina de Programação Estruturada. Ao longo do relatório, procuramos não apenas descrever o que foi feito, mas também discutir as decisões de projeto, os fundamentos teóricos e os resultados experimentais.

Acreditamos que o projeto atingiu seus objetivos principais:

- Implementamos corretamente as cinco operações aritméticas fundamentais, validadas por uma suíte abrangente de testes.
- Desenvolvemos estruturas de dados eficientes (array dinâmico e `BigInteger`) com gerenciamento cuidadoso de memória.
- Implementamos algoritmos clássicos como multiplicação escolar, Karatsuba e *binary splitting* para fatoriais.
- Criamos uma interface flexível que suporta modos interativo e batch.
- Realizamos uma análise experimental detalhada do desempenho no ambiente Google Colab, confirmando as complexidades teóricas esperadas.
- Introduzimos a busca binária como ferramenta para investigar limites computacionais, com aplicação ao cálculo de fatorial.

5.2 Lições Aprendidas

O desenvolvimento deste projeto proporcionou aprendizados valiosos:

1. **A importância da modularidade:** A organização em camadas e módulos facilitou enormemente o desenvolvimento, teste e depuração. Cada componente pôde ser desenvolvido e validado isoladamente antes da integração.
2. **O valor dos testes automatizados:** A suíte de testes nos deu confiança para fazer modificações no código, sabendo que qualquer erro seria detectado rapidamente. A abordagem de gerar resultados esperados com Python (que já possui precisão arbitrária) provou-se particularmente eficaz.

3. **A complexidade do gerenciamento de memória:** Em C, o gerenciamento manual de memória exige cuidado constante e disciplina rigorosa. Aprendemos a importância de liberar recursos em todos os caminhos de erro e de projetar estruturas que facilitem a liberação correta.
4. **A riqueza dos algoritmos de precisão arbitrária:** O tema conecta conceitos fundamentais de matemática, algoritmos, estruturas de dados e arquitetura de computadores. A implementação do algoritmo de Karatsuba, em particular, foi um exercício valioso de recursão e divisão e conquista.
5. **A importância da experimentação:** As medições de desempenho revelaram comportamentos que não seriam evidentes apenas pela análise teórica, como os picos de memória no cálculo de fatorial e o comportamento linear da divisão no ambiente testado.
6. **Ambientes de desenvolvimento em nuvem:** O Google Colab provou-se uma plataforma adequada para experimentação científica, oferecendo recursos computacionais suficientes e reprodutibilidade.

5.3 Trabalhos Futuros

Diversas direções para trabalhos futuros podem ser exploradas:

1. **Otimizações algorítmicas:** Implementar algoritmos mais avançados como Toom-Cook, multiplicação baseada em FFT (Schönhage-Strassen) e divisão de Burnikel-Ziegler para melhorar o desempenho em números muito grandes.
2. **Representação binária:** Migrar para representação em base 2^{32} ou 2^{64} , melhorando significativamente a eficiência e reduzindo o consumo de memória.
3. **Paralelização:** Explorar paralelismo em múltiplos núcleos para operações com números muito grandes, particularmente para multiplicação e fatorial.
4. **Operações adicionais:** Implementar exponenciação, raiz quadrada, máximo divisor comum, potenciação modular e outras operações úteis para aplicações criptográficas e matemáticas.
5. **Interface Python:** Criar bindings para Python, permitindo uso mais amplo da biblioteca e facilitando a integração com o ecossistema científico Python.
6. **Análise mais aprofundada:** Realizar medições com mais amostras, explorar limites superiores de n para cada operação, investigar a variabilidade entre diferentes configurações de hardware e estudar o comportamento de *cache*.
7. **Otimização de memória:** Investigar estratégias para reduzir os picos de memória observados no cálculo de fatorial, possivelmente através de diferentes estratégias de ordenação das multiplicações ou uso de alocação sob demanda.

5.4 Disponibilidade do Código

O código fonte completo da biblioteca `add-masterc` está disponível sob licença MIT no repositório GitHub:

`https://github.com/luisr-1/add-masterc.git`

O repositório contém:

- Código fonte completo da biblioteca
- Sistema de testes automatizados
- Scripts para geração de resultados esperados
- Documentação básica de uso
- Exemplos de aplicação

Esperamos que este trabalho possa servir como referência para outros estudantes interessados em explorar o fascinante mundo da aritmética de precisão arbitrária e que o código disponível possa ser útil para fins educacionais e de pesquisa.

Agradecimentos

Arthur Tembikoski: Agradecimento especial a minha mulher Leticia, pois sem ela nada disso seria possível e a meus parentes e amigos.

Luis Rodrigues:

Este trabalho é fruto de uma jornada que não se fez sozinha. Reservamos este espaço para expressar nossa gratidão àqueles que, direta ou indiretamente, contribuíram para que ele se tornasse realidade.

Aos professores da Universidade Federal do ABC, que ao longo desta caminhada não apenas transmitiram conhecimento, mas também inspiraram curiosidade intelectual e rigor científico. Aqueles que nos desafiaram com questões difíceis, que exigiram mais do que achávamos ser capazes, que nos mostraram que o aprendizado verdadeiro ocorre na fronteira entre o que sabemos e o que ainda podemos descobrir.

Aos colegas de curso, que compartilharam esta caminhada, tornando os momentos difíceis mais leves e as conquistas mais significativas.

À comunidade de software livre, que nos lembra diariamente que o conhecimento deve ser compartilhado e que a tecnologia pode ser uma ferramenta de transformação social.

Por fim, à universidade pública, gratuita e de qualidade, representada pela UFABC. Que possamos sempre defender este patrimônio da sociedade brasileira, formando cidadãos críticos e profissionais competentes.

References

- [1] KNUTH, Donald E. *The Art of Computer Programming, Volume 2: Seminumerical Algorithms*. 3. ed. Boston: Addison-Wesley, 1997.
- [2] CORMEN, Thomas H. et al. *Introduction to Algorithms*. 3. ed. Cambridge: MIT Press, 2009.

- [3] GRANLUND, Torbjörn; GMP Development Team. *GNU MP: The GNU Multiple Precision Arithmetic Library*. 2015. Disponível em: <https://gmplib.org/>. Acesso em: 10 mar. 2025.
- [4] KARATSUBA, Anatolii; OFMAN, Yuri. Multiplication of multidigit numbers on automata. *Soviet Physics-Doklady*, v. 7, p. 595–596, 1963.
- [5] SCHÖNHAGE, Arnold; STRASSEN, Volker. Schnelle Multiplikation großer Zahlen. *Computing*, v. 7, n. 3-4, p. 281–292, 1971.
- [6] BURNIKEL, Christoph; ZIEGLER, Joachim. *Fast Recursive Division*. Max-Planck-Institut für Informatik, Relatório Técnico MPI-I-98-1-022, 1998.